ROAR Solo Mission

Technical Documentation

Part 2 - Simulation on ROS and Gazebo

Prepared by:

Saif Elden Mahmoud

Date:

April 29, 2026

Contents

1 Task 2: Simulation on ROS and Gazebo **3**
1.1 Documentation-Driven Workflow 3
1.2 AI-Assisted Development and Advanced Tooling 4
1.3 System Architecture and Productivity Enhancements 4
1.4 Implemented System Building Blocks 5
1.5 Python Dependency and Transform Compatibility 5
1.6 Robot Modeling and Design Challenges 5
1.7 Environment Setup and Simulation Launch 6
1.8 Runtime Verification and Topic Validation 6
1.9 TF Tree and RViz Debugging 6
1.10 ArUco Marker Detection and Tracking Node 7
1.11 Visualizing Output with GUI and RViz2 8
1.12 Additional Photos Showcasing the Working Project 8

List of Figures

1 Gazebo simulation with the robot spawned in the marker world and the teleop GUI used during runtime testing. 6
2 Terminal verification showing active nodes, bridged topics, and ArUco marker detection output with ID 53. 7
3 TF tree exported from the running system after joint-state publication was restored. 7
4 RViz model view after the transform issue was fixed and the wheel frames resolved correctly. 8
5 Integrated system overview showing topic checks, node activity, ArUco detection output, RViz, Gazebo, and the map context in one capture. . . 9
6 Additional photos showcasing the working project. 9
7 Additional photos showcasing the working project. 9

1 Task 2: Simulation on ROS and Gazebo

This section documents the execution and methodology for the ROS 2 and Gazebo simulation tasks, with emphasis on the supporting documentation, debugging workflow, and ArUco Marker Detection and Tracking.

1.1 Documentation-Driven Workflow

The report and implementation were guided by the steering files stored in `.kiro/steering`. `eng.md` kept the work disciplined by enforcing explicit assumptions, minimal edits, and verifiable changes. `ros2-gazebo-best-practices.md` aligned the project with the correct ROS 2 topic names, TF frame names, xacro structure, launch patterns, and bridge conventions. `latex-workflow.md` defined the report structure and the compile-after-edit workflow that kept the documentation consistent.

- **Camera integration notes:** `CAMERA_INTEGRATION_TUTORIAL.md` captured the camera fixes that mattered most: preserving the fixed joint so Gazebo would not lump the camera link away, pushing the camera outside the robot body to avoid clipping, keeping the sensor frame aligned with `camera_link`, and remapping the Gazebo camera feed when it was publishing on the wrong scoped topic instead of the ROS 2 topics used by perception.
- **Camera validation notes:** `CAMERA_TESTING_GUIDE.md` defined the verification path using `ros2 topic list`, `ros2 topic hz`, RViz, `rqt_image_view`, and `verify_camera.py`.

Shell Aliases from `~/.bashrc`: Rather than referencing a separate alias note, I used the real aliases from my shell startup file. The most useful ones for this project were:

- `rosb`: builds the workspace and re-sources the install space, which keeps the environment consistent after code changes.
- `rosd`: launches the description in RViz for fast visual inspection of the robot model.
- `rosg` and `rosgp`: restart Gazebo and spawn the robot or marker world quickly, which reduces stale-session issues during debugging.
- `rosgu`: opens the teleop GUI for manual control during testing.
- `rosar`: launches the ArUco perception stack.
- `ssb`, `alis`, and `aliss`: source the workspace or reload the shell configuration without repeating long commands.

Strengths of using these aliases:

- They cut startup time by turning multi-command launch sequences into one-command shortcuts.
- They reduce typing mistakes when rebuilding, sourcing, and relaunching the workspace repeatedly.
- They make it easier to restart the simulation cleanly when Gazebo or ROS starts from a stale session.
- They keep debugging reproducible because the same launch path is used every time.

1.2 AI-Assisted Development and Advanced Tooling

To accelerate the development, debugging, and deployment of this complex robotic simulation, an advanced AI-assisted workflow was integrated directly into the methodology.

Custom Steering and Context (MCPs): To guide the AI agents, I relied on the steering files above and on ROS-focused Model Context Protocol (MCP) servers to give the tools live workspace context. This kept the generated code and debugging suggestions aligned with the simulation stack instead of generic ROS examples.

Specialized AI Agents: I leveraged three distinct AI agents, deploying each according to its unique specialization for debugging and writing complex system parts:

- **VS Code Copilot:** Utilized primarily for inline code generation and auto-completing ROS 2 boilerplate from the editor.
- **Kiro Agent:** Deployed specifically for tracking and resolving complex multi-file architectural bugs.
- **Bonsai AI Agent:** Employed for high-level simulation tuning, system-level design, and optimizing ROS 2 control architectures.

Strategic Model Selection: Different foundation models were strategically queried based on their specific strengths to tackle various sub-components of the mission:

- **Sonnet 4.5:** Used for its sheer capability in writing clean, well-structured ROS 2 code and handling complex node definitions.
- **Gemini 3.1:** Leveraged for its profound logical reasoning during deep debugging sessions and Gazebo environment troubleshooting.
- **GPT-5.4 Mini:** Utilized for rapid iteration, quick documentation generation, and writing lighter scripts with extremely low latency.

1.3 System Architecture and Productivity Enhancements

To ensure scalability and clean organization, the workspace was decoupled into a highly modular file architecture.

Workspace Structure:

- `ros2_ws/src/` contains specialized packages: `my_robot_description` for the URDF models, `my_robot_gazebo` for simulation environments, `my_robot_perception`, and the `aruco_tracking` node.
- `Scripts/` directory offloads validation and environment configuration from the ROS source, utilizing bash and python scripts such as `setup_environment.sh`, `verify_marker_world.sh`, and `validate_geometry.py`.
- **Standalone Utilities:** I developed a `robot_teleop_gui.py` graphical interface alongside standard ROS nodes to effectively drive and test the robot within Gazebo.

Productivity Enhancements: To accelerate debugging and deployment, I used the shell aliases from `/.bashrc`. These shortcuts significantly cut down the time required to source environments, rebuild the workspace, and launch the Python-based simulation files during testing.

1.4 Implemented System Building Blocks

The report is strongest when it focuses on the pieces that are already implemented and validated in the workspace.

- **Robot description:** `my_robot.urdf.xacro` parameterizes the chassis, suspension arms, wheels, IMU, and camera position so the model can be tuned without rewriting geometry from scratch.
- **Gazebo configuration:** `gazebo.xacro` applies friction values, IMU noise, camera sensor settings, and the fixed-joint preservation needed to keep the camera link alive in simulation.
- **Simulation launch flow:** `spawn_robot.launch.py` loads the marker world, spawns the robot, bridges the scoped Gazebo camera and IMU topics, and publishes the static camera transform.
- **Perception launch flow:** `aruco_detection.launch.py` launches the existing `aruco_tracking` node with the correct dictionary size, marker scale, simulation time, and camera topic remapping.
- **Operator control and verification:** `robot_teleop_gui.py` publishes `/cmd_vel` from a GUI, while `verify_imu.py` confirms the IMU topic is actually publishing useful data.
- **Visualization split:** `display.launch.py` and `rviz_only.launch.py` separate pure visualization from the full simulation loop, which makes testing faster and less fragile.

1.5 Python Dependency and Transform Compatibility

The ArUco pose pipeline depends on quaternion and matrix transforms, so the Python environment had to be compatible with the code path used by `aruco_tracking`. During development, an older `numpy` release removed the legacy `np.float` alias used by the transform math, and the `tf_transformations` package was not initially available in the environment. I solved this by keeping the compatibility guard in `aruco_node.py`, declaring `tf_transformations` as a package dependency, and rebuilding the workspace so pose publishing could run normally.

1.6 Robot Modeling and Design Challenges

Developing the physical simulation description presented significant kinematic challenges. I encountered distinct problems properly aligning the robot's linkages within the Xacro specifications.

Iterative Offset Strategy & Camera Issues: A particularly difficult issue arose when placing the camera sensor; initially, the camera link spawned directly *inside* the robot's main chassis block, resulting in obscured vision and complete failure of the simulated camera feed.

To resolve this, I refactored the URDF models to use parametric Xacro offsets across the entire design. This engineering choice allowed me to programmatically alter link translations in real-time, pulling the camera safely outside the chassis body to acquire a clear field-of-view (FOV) without breaking the robot's geometry. I also preserved the

fixed camera joint and kept the sensor frame aligned with `camera_link` so the camera feed remained valid in RViz and the TF tree.

1.7 Environment Setup and Simulation Launch

The first step in fulfilling this task involved properly setting up the workspace and launching the robot simulation in Gazebo using Python-based `.py` launch files to simultaneously start Gazebo, upload the robot description, and spawn the ArUco markers. The world file was referenced explicitly so Gazebo would load `marker_world.world` instead of silently reusing an older cached world or model state.

Methodology: I utilized the provided `my_robot_gazebo` and `my_robot_description` packages to spawn the robot within a designated world containing the ArUco marker poles.

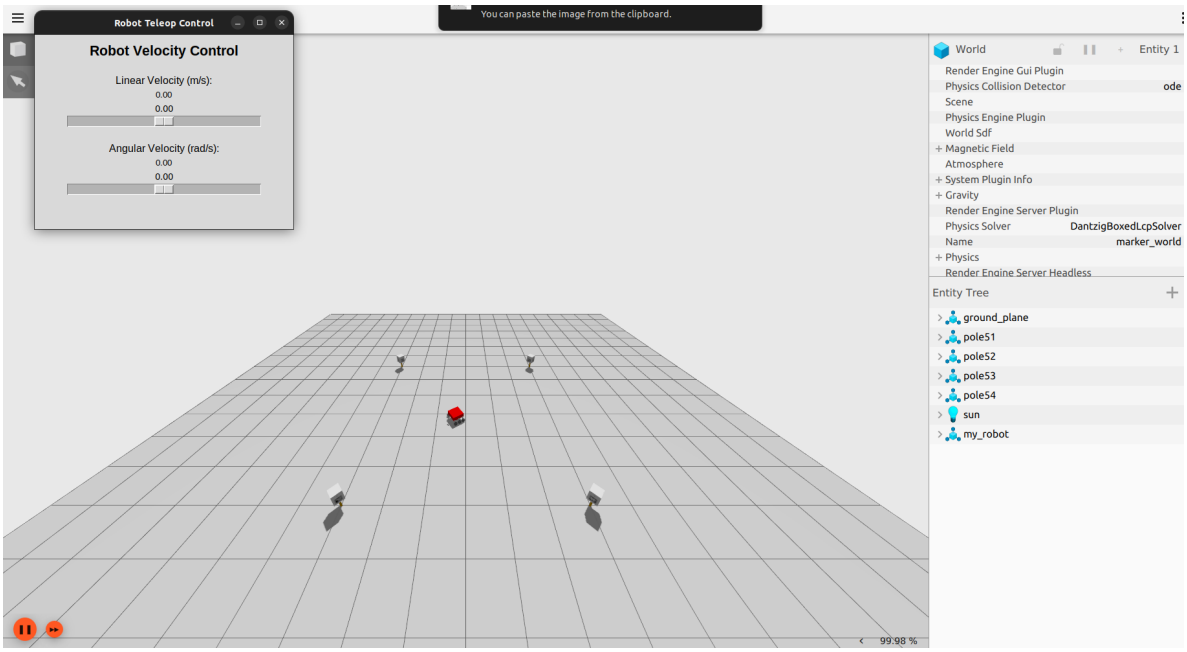


Figure 1: Gazebo simulation with the robot spawned in the marker world and the teleop GUI used during runtime testing.

1.8 Runtime Verification and Topic Validation

The terminal evidence is valuable because it confirms that the expected runtime graph came up cleanly: the robot state publisher, camera frame publisher, bridge nodes, RViz, and the ArUco detection node all launched together. The topic listing also shows that the perception pipeline is connected to the correct ROS names and that the detected marker data is flowing through the system.

1.9 TF Tree and RViz Debugging

One of the more visible problems during validation was that the RViz model looked corrupted: the wheel links appeared stacked and RViz reported missing transforms for parts of the robot. The investigation showed that joint-state propagation was the weak point in the simulation pipeline, so the wheel and arm links were not being updated

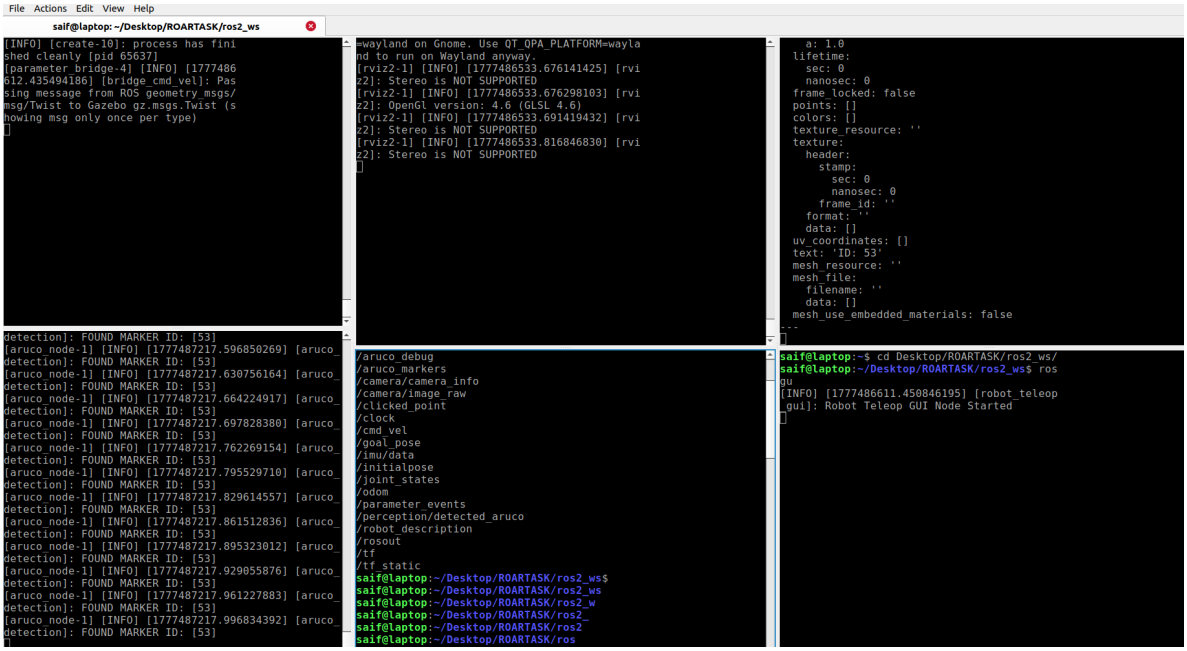


Figure 2: Terminal verification showing active nodes, bridged topics, and ArUco marker detection output with ID 53.

reliably. Once joint-state publication and the transform chain were restored, the model tree became complete and RViz could resolve the links correctly.

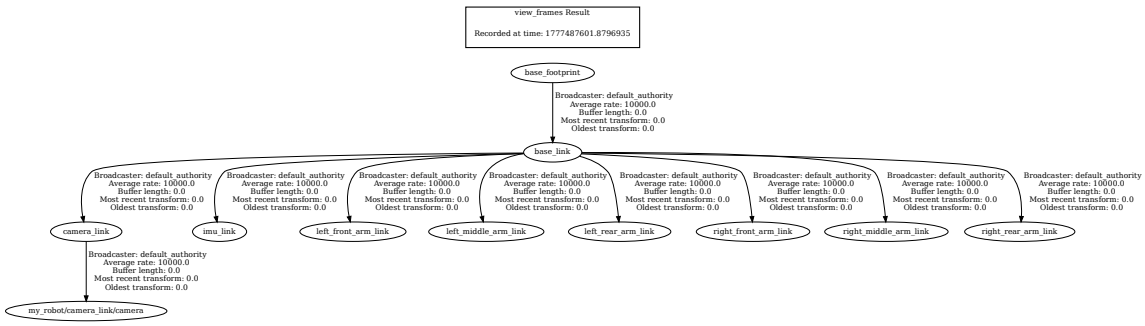


Figure 3: TF tree exported from the running system after joint-state publication was restored.

1.10 ArUco Marker Detection and Tracking Node

The core component of this task is the `aruco_tracking` node, responsible for real-time identification of ArUco markers from the robot's camera feed.

Methodology: I integrated the existing ArUco tracking stack with the robot camera feed through the launch and bridge configuration, so the marker results appear in the ROS tools without repeating the detector implementation itself in the report.

Technical Highlights & Code Strengths:

- **Robust Message Conversion:** Seamless integration with `cv_bridge` to confidently transfer data between ROS `Image` messages and image-processing data paths.

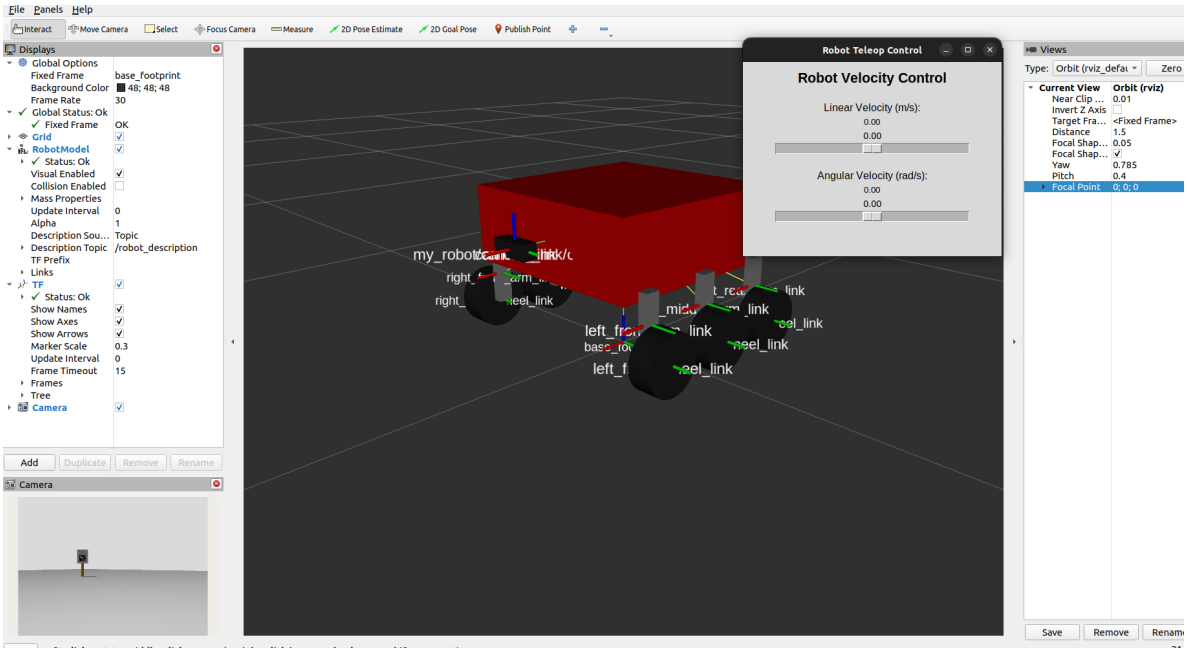


Figure 4: RViz model view after the transform issue was fixed and the wheel frames resolved correctly.

- **Dynamic Pose Estimation:** Accurate 3D pose estimation utilizing camera intrinsic matrices to provide continuous feedback of the markers' relative positions.
- **Clean System Architecture:** A highly modular Object-Oriented paradigm ensuring the node can be scaled and easily maintained.

Rather than repeating the detection algorithm itself, the report focuses on the implemented integration around it. The perception stack is launched by `aruco_detection.launch.py`, which starts the existing `aruco_tracking` node, sets `arucoDictType=7` and `markerSize=0.15`, and remaps the robot camera topics into the names the node expects. That makes the report reflect the actual engineering work in the workspace: wiring the detector into the simulation, not re-explaining the detection task that was already provided.

1.11 Visualizing Output with GUI and RViz2

To validate the node's performance, I implemented visualization techniques to display both the raw and annotated camera feeds, proving that the tracking system works successfully under simulated conditions. The validation flow followed the camera testing guide by checking `/camera/image_raw`, `/camera/camera_info`, RViz, `rqt_image_view`, and the camera verification script.

Execution Details: I utilized `rqt_image_view` and a custom `robot_teleop_gui.py` to ensure both manual control of the rover and real-time visualization of the tracking subsystem were successful. The implementation effectively handles varying lighting conditions and distances within the Gazebo engine.

1.12 Additional Photos Showcasing the Working Project

These extra captures provide additional evidence that the simulation, RViz, and perception pipeline were all operating together correctly.

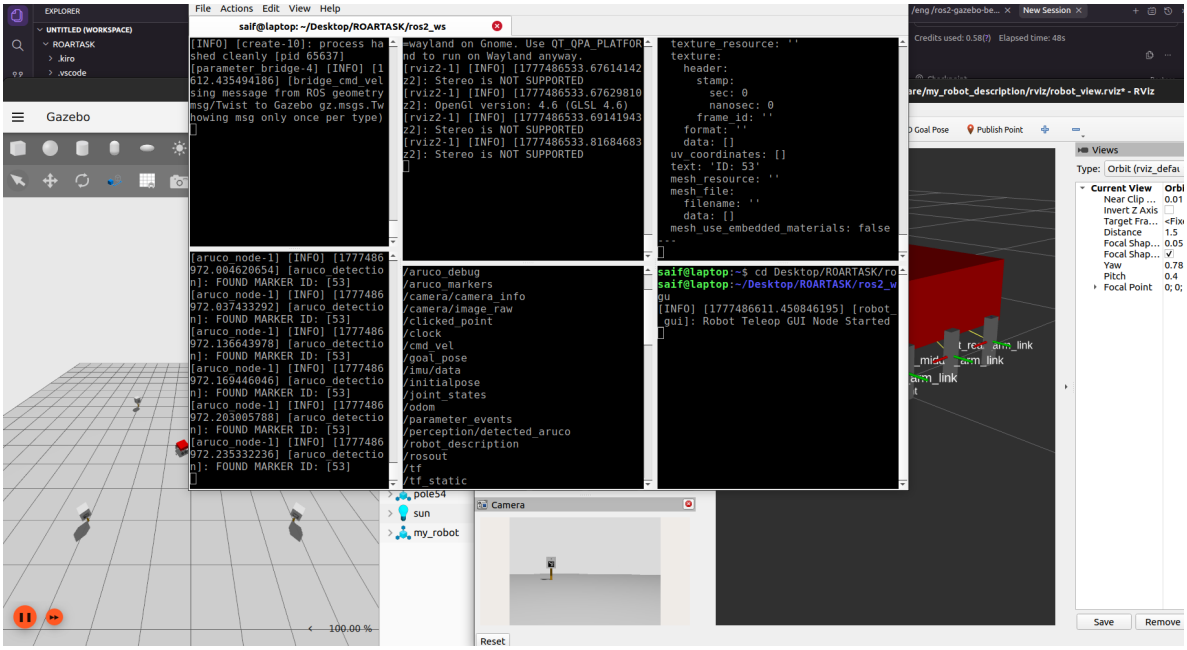
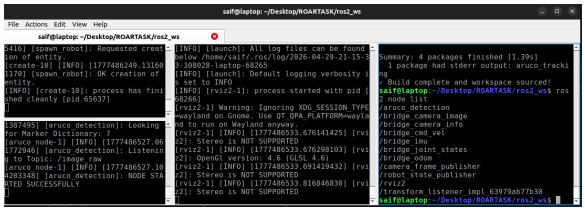


Figure 5: Integrated system overview showing topic checks, node activity, ArUco detection output, RViz, Gazebo, and the map context in one capture.



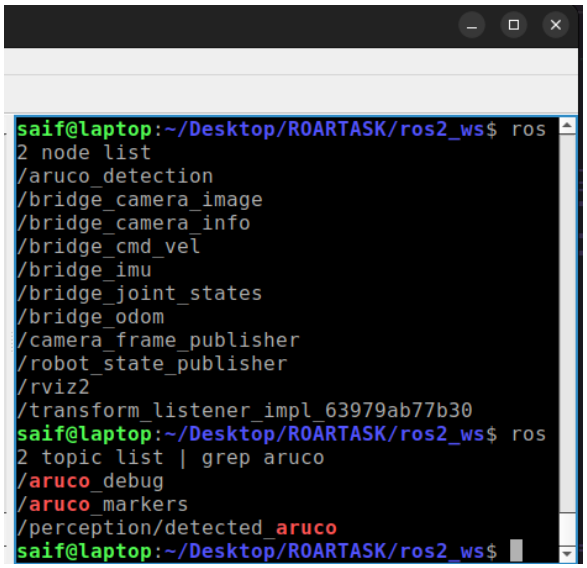
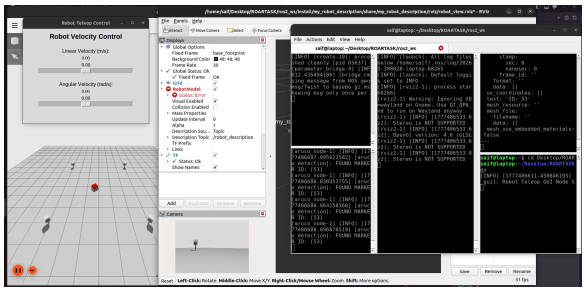


Figure 6: Additional photos showcasing the working project.



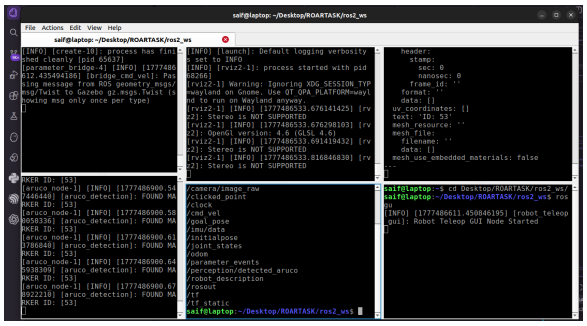


Figure 7: Additional photos showcasing the working project.